

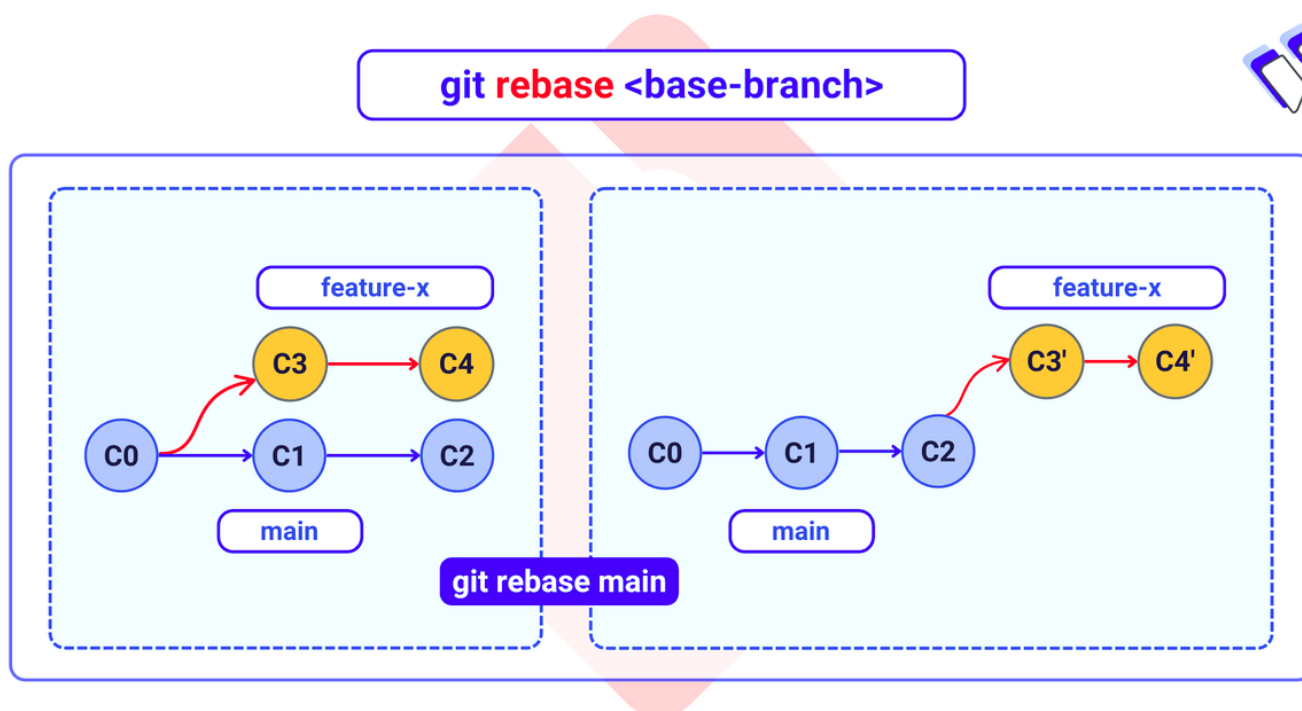
Ferramentas Avançadas

Nessa aula vamos apresentar algumas ferramentas adicionais para o processo de desenvolvimento usando Git. Nela iremos ver ferramentas de monitoramento do repositório, diferentes formas de atualizá-lo e, por fim, como lidar com erros.

Atualizando o repositório

No Git, existem duas maneiras principais de integrar as mudanças de um branch para outro: o merge e o rebase. O `git rebase` é usado para integrar alterações de um branch para outro, movendo ou reaplicando uma série de commits sobre um novo commit base. Esse processo reescreve o histórico do projeto, resultando em um histórico de commits mais limpo e linear, em oposição ao commit de mesclagem de três vias criado pelo comando `git merge`.

Ele é usado principalmente quando você está desenvolvendo uma branch com uma feature nova e a `main` recebe novas atualizações se tornando um projeto estável. Então deseja-se atualizar a sua branch para que ela acompanhe a nova `main`, garantindo um merge mais suave no futuro. Evite fazer rebase em branches que já foram compartilhadas com outras pessoas, pois o rebase reescreve o histórico e altera os hashes dos commits



```
git switch feature-x # muda para branch feature-x
git rebase main      # muda a base de branch feature-x para a nova main
```

Obs: O rebase é baseado no seu repositório local. Então, se deseja dar um rebase baseado na branch `main` mais atualizada, é importante atualizar a branch `main`. Utiliza-se `git switch main` e depois `git pull`."

Monitorando alterações

Nessa seção vamos apresentar alguns comandos que facilitam monitorar as alterações feitas em um repositório.

Verificar mudanças no working directory

O comando `git status` mostra o estado atual do repositório, indicando quais arquivos foram modificados no working directory em relação ao último commit da branch atual. Além disso, ele também informa quais arquivos estão na staging area, ou seja, prontos para serem commitados, quais ainda não foram adicionados com `git add` e quais arquivos não rastreados existem no diretório.

```
(base) joao@dell-pc:~/Documents/repo/meu_primeiro_repo$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        meu_codigo.py

(base) joao@dell-pc:~/Documents/repo/meu_primeiro_repo$
```

Listando commits anteriores:

Podemos listar todo o histórico de commits de um repositório usando o `git log`. Nele conseguimos ver informações importantes sobre cada commit, seu autor, quando foi feito, os comentários e o hash do commit.

```
joao@dell-pc: ~/Documents/repo/Sonho 91x45
(base) joao@dell-pc:~/Documents/repo/Sonho$ git log
commit 1f7ff1527f3bb401ed5bf14de65089eb601b3861 (HEAD -> git, origin/git)
Author: JPBG-USP <joao_garciajp@usp.br>
Date: Mon Feb 16 11:11:37 2026 -0300

    Adicionando a config de nome da branch default na aula

commit 5c36bd4e5f14c7dc23c660ed4af5a5c058ae47c0
Author: JPBG-USP <joao_garciajp@usp.br>
Date: Thu Feb 12 17:16:01 2026 -0300

    Adicionando material do Learning Branches

    Nesse commit adicionei na aula de Criando Branches o material do site Learning Branches
    como material auxiliar.

commit d4d8dbe3b33670984ce9efd1f0fe778a6b5211f1
Author: JPBG-USP <joao_garciajp@usp.br>
Date: Tue Feb 10 13:26:05 2026 -0300

    Adicionando aula criando branches

    Nesse commit adicionei os arquivos da aula de criando branch, ainda falta alguns detalh
    es sobre a secao fazendo juntos.

commit 0945bdbcee639f3927dcb4c7f9609ef89b71cb2e
Author: JPBG-USP <joao_garciajp@usp.br>
Date: Tue Feb 10 10:33:14 2026 -0300

    Adicionando novas secoes na aula 1 de git

    Apos reuniao com Toms e Tim vimos detalhes que precisavam ser adicionados e alguns que
    havia esquecido. Com isso adicionei as secoes de git clone e push na aula 1.

commit 09a6c74b4c5d74082215e3200b0e600e3d02
```

Abaixo temos a imagem destacando cada informação:

```
joao@dell-pc: ~/Documents/repo/Sonho 91x45
(base) joao@dell-pc:~/Documents/repo/Sonho$ git log
commit 1f7ff1527f3bb401ed5bf14de65089eb601b3861 (HEAD -> git, origin/git)
Author: JPBG-USP <joao_garciajp@usp.br>
Date: Mon Feb 16 11:11:37 2026 -0300
    Hash do commit

    Adicionando a config de nome da branch default na aula

commit 5c36bd4e5f14c7dc23c660ed4af5a5c058ae47c0
Author: JPBG-USP <joao_garciajp@usp.br>
Date: Thu Feb 12 17:16:01 2026 -0300
    Comentário do commit

    Adicionando material do Learning Branches

    Nesse commit adicionei na aula de Criando Branches o material do site Learning Branches
    como material auxiliar.

commit d4d8dbe3b33670984ce9efd1f0fe778a6b5211f1
Author: JPBG-USP <joao_garciajp@usp.br>
Date: Tue Feb 10 13:26:05 2026 -0300
    Autor e data

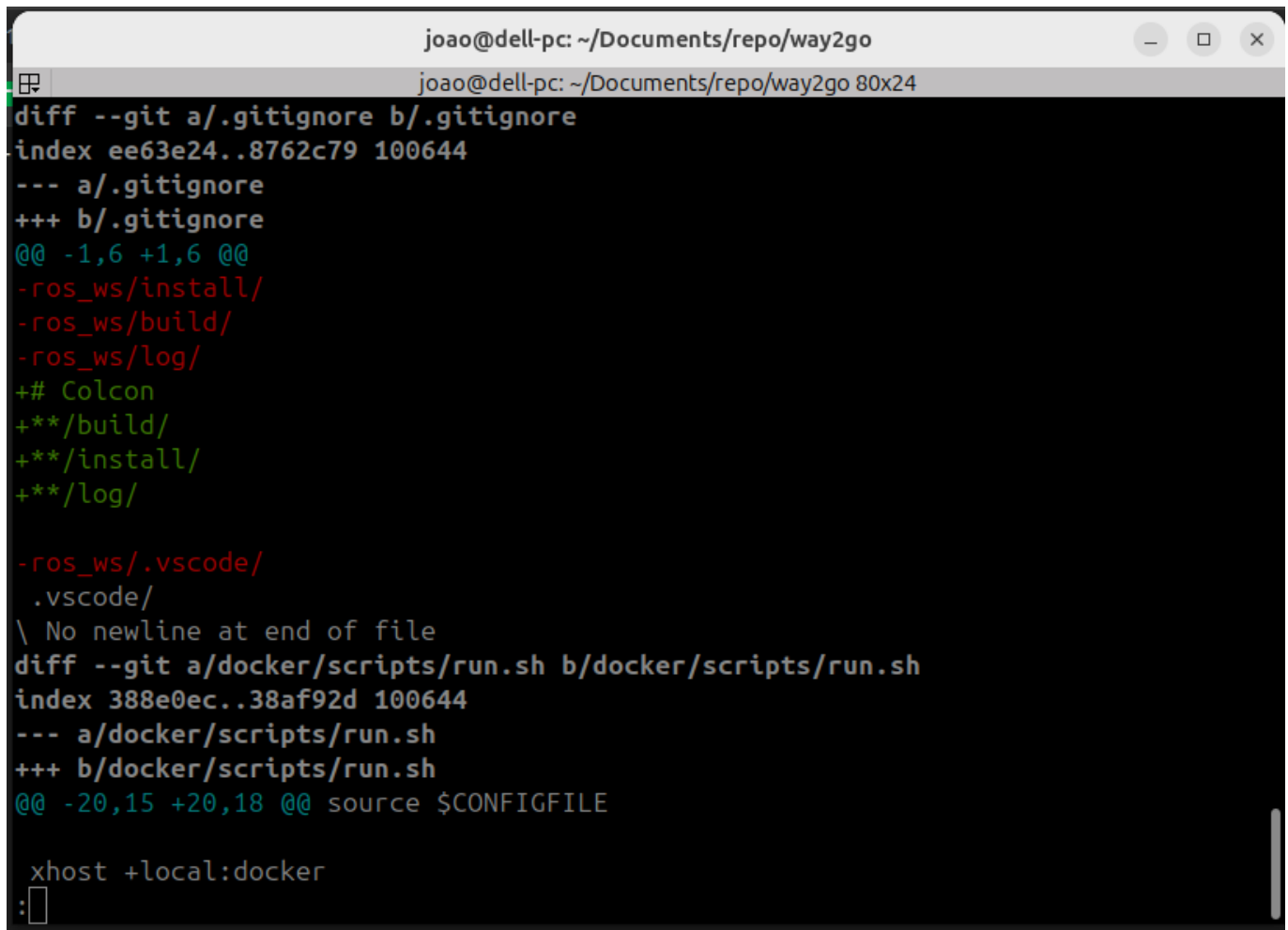
    Adicionando aula criando branches

    Nesse commit adicionei os arquivos da aula de criando branch, ainda falta alguns detalhes
    sobre a seção fazendo juntos.

commit 0945bdbcee639f3927dcb4c7f9609ef89b71cb2e
Author: JPBG-USP <joao_garciajp@usp.br>
Date: Tue Feb 10 10:33:14 2026 -0300
```

Verificando modificações

O comando `git diff` é uma ferramenta Git multifuncional usada para exibir as diferenças entre vários estados de um repositório, como entre o diretório de trabalho, a área de preparação (staging area), commits e branches.



```
joao@dell-pc: ~/Documents/repo/way2go
joao@dell-pc: ~/Documents/repo/way2go 80x24
diff --git a/.gitignore b/.gitignore
index ee63e24..8762c79 100644
--- a/.gitignore
+++ b/.gitignore
@@ -1,6 +1,6 @@
-ros_ws/install/
-ros_ws/build/
-ros_ws/log/
+# Colcon
+**/build/
+**/install/
+**/log/

-ros_ws/.vscode/
.vscode/
\ No newline at end of file
diff --git a/docker/scripts/run.sh b/docker/scripts/run.sh
index 388e0ec..38af92d 100644
--- a/docker/scripts/run.sh
+++ b/docker/scripts/run.sh
@@ -20,15 +20,18 @@ source $CONFIGFILE

xhost +local:docker
:
```

Ele acaba não sendo muito utilizado pelo fato de as informações apresentadas por ele não serem muito visuais. Hoje com extensões no VScode algumas dessas ferramentas ficam de lado. Mas ele continua sendo muito útil para ambientes sem interface gráfica.

OBS IMPORTANTE: Tanto o git log quanto o `git diff` é possível rodar a lista utilizando seta para baixo, para sair não utiliza `Ctrl+Z`, e sim pressionar a tecla `q`

Encontrando o culpado

Em projetos maiores, é comum surgir a seguinte dúvida: quem modificou determinada linha de código e quando isso foi feito? Para responder a esse tipo de pergunta, utilizamos o comando `git blame`.

Diferente de comandos como `git log`, que mostram o histórico de commits como um todo, o `git blame` foca em um arquivo específico e detalha a origem de cada linha existente na versão atual.

```
git blame nome_do_arquivo.cpp
```

```

joao@dell-pc: ~/Documents/repo/way2go
joao@dell-pc: ~/Documents/repo/way2go 140x40
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 32)
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 33) hardware_interface::return_type write(
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 34) const rclcpp::Time & time,
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 35) const rclcpp::Duration & period) override;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 36)
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 37) private:
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 38) std::unique_ptr<SerialBridge> comms_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 39)
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 40) // Config
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 41) std::string port_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 42) int baudrate_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 43) SerialBridgeMsg::DriveMode robot_mode_;
54c9db57 (JPBG-USP 2026-02-17 18:12:40 -0300 44) std::map<std::string, size_t> joint_map_;
54c9db57 (JPBG-USP 2026-02-17 18:12:40 -0300 45) int num_joints = {0};
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 46)
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 47) // Joint Data
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 48) std::vector<double> hw_positions_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 49) std::vector<double> hw_velocities_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 50) std::vector<double> hw_efforts_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 51) std::vector<double> hw_commands_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 52)
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 53) // IMU Data Buffers (Allocated for 3 axes: X, Y, Z)
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 54) std::vector<double> imu_acc_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 55) std::vector<double> imu_gyro_;
54c9db57 (JPBG-USP 2026-02-17 18:12:40 -0300 56) std::vector<double> imu_orientation_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 57)
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 58) // Orientation state
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 59) double roll_ = 0.0;
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 60) double pitch_ = 0.0;
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 61) double yaw_ = 0.0;
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 62)
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 63) // Complementary filter coefficient
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 64) double alpha_ = 0.98;
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 65)
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 66) // Time handling
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 67) rclcpp::Time last_time_;
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 68) bool first_update_ = true;
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 69)
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 70) void updateOrientation();

```

Conseguimos ver dois diferentes commits e também podemos ver as alterações que ainda não foram commitadas. Veja a imagem abaixo com as indicações:

```

joao@dell-pc: ~/Documents/repo/way2go
joao@dell-pc: ~/Documents/repo/way2go 140x40
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 32)
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 33) hardware_interface::return_type write(
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 34) const rclcpp::Time & time,
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 35) const rclcpp::Duration & period) override;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 36)
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 37) private:
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 38) std::unique_ptr<SerialBridge> comms_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 39)
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 40) // Config
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 41) std::string port_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 42) int baudrate_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 43) SerialBridgeMsg::DriveMode robot_mode_;
54c9db57 (JPBG-USP 2026-02-17 18:12:40 -0300 44) std::map<std::string, size_t> joint_map_;
54c9db57 (JPBG-USP 2026-02-17 18:12:40 -0300 45) int num_joints = {0};
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 46)
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 47) // Joint Data
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 48) std::vector<double> hw_positions_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 49) std::vector<double> hw_velocities_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 50) std::vector<double> hw_efforts_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 51) std::vector<double> hw_commands_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 52)
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 53) // IMU Data Buffers (Allocated for 3 axes: X, Y, Z)
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 54) std::vector<double> imu_acc_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 55) std::vector<double> imu_gyro_;
54c9db57 (JPBG-USP 2026-02-17 18:12:40 -0300 56) std::vector<double> imu_orientation_;
b6b5aa20 (JPBG-USP 2026-02-17 01:49:04 -0300 57)
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 58) // Orientation state
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 59) double roll_ = 0.0;
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 60) double pitch_ = 0.0;
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 61) double yaw_ = 0.0;
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 62)
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 63) // Complementary filter coefficient
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 64) double alpha_ = 0.98;
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 65)
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 66) // Time handling
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 67) rclcpp::Time last_time_;
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 68) bool first_update_ = true;
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 69)
00000000 (Not Committed Yet 2026-02-24 12:47:13 -0300 70) void updateOrientation();

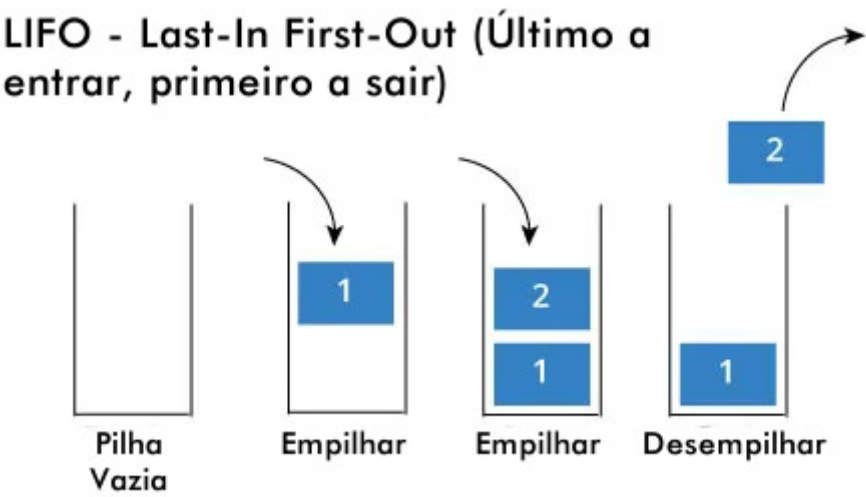
```

- Azul, o commit mais antigo
- Verde, um commit mais recente
- Amarelo, mudanças ainda não commitadas

Salvamento temporário

O comando `git stash` salva temporariamente suas alterações não commitadas (tanto as staged quanto as unstaged, para tracked files) em uma pilha "stash" local, permitindo que você retorne a um diretório de trabalho limpo. Ou seja, simplificadaamente você guarda as suas modificações temporariamente em uma pilha. O comando `git stash pop` retorna essas alterações dessa pilha temporária. Por padrão, o `git stash` salva apenas alterações em arquivos já rastreados pelo Git.

Nessa parte é importante relembrar o conceito de pilha como estrutura de dados na programação. Ela é uma estrutura de dados, uma forma de organizar informações onde a ordem importa. No caso da pilha ela segue o que chamamos de LIFO (Last-In, First Out), ou seja, o ultimo dado a entrar na pilha é o primeiro a sair.



Nessa imagem o empilhar é o `git stash` e o desempilhar é o `git pop`. Outra analogia muito utilizada é a de lavar uma pilha de pratos. Você primeiro empilha eles e quando você for lavar, você lava o topo da pilha primeiro.

O `git stash` funciona de forma parecida com o `git add`. Ao lado dele você pode passar o caminho dos arquivos que deseja colocar no salvamento temporário.

```
git stash <path-to-file>
```

Segue abaixo a lista dos comandos usando stash:

Comando / Parâmetro	Descrição
<code>git stash</code>	Salva as modificações rastreadas do working directory e limpa a área de trabalho.
<code>git stash push</code>	Forma explícita do <code>stash</code> , permite usar opções adicionais.
<code>git stash pop</code>	Aplica o stash mais recente e o remove da lista.

Comando / Parâmetro	Descrição
<code>git stash -u</code> ou <code>--include-untracked</code>	Inclui arquivos não rastreados no stash.
<code>git stash -a</code> ou <code>--all</code>	Inclui arquivos não rastreados e ignorados.
<code>git stash save "mensagem"</code>	Forma antiga para salvar com mensagem descritiva.
<code>git stash -m "mensagem"</code>	Adiciona uma mensagem ao stash para facilitar identificação.
<code>git stash list</code>	Lista todos os stashes salvos.
<code>git stash show</code>	Mostra um resumo das alterações do stash mais recente.
<code>git stash show -p</code>	Mostra o diff completo do stash.
<code>git stash apply</code>	Aplica o stash mais recente sem removê-lo da lista.
<code>git stash apply stash@{n}</code>	Aplica um stash específico.
<code>git stash drop stash@{n}</code>	Remove um stash específico.
<code>git stash clear</code>	Remove todos os stashes.
<code>git stash branch nome-branch</code>	Cria uma nova branch a partir do commit onde o stash foi criado e aplica o stash nela.

Errar é humano

Restaurando arquivos

Durante o desenvolvimento é comum modificarmos arquivos por engano, adicionarmos algo à staging area sem querer ou simplesmente querermos voltar um arquivo ao estado anterior. Para esses casos, o Git disponibiliza o comando `git restore`.

O `git restore` é utilizado para restaurar arquivos no working directory ou na staging area, desfazendo alterações locais. Ele não altera o histórico de commits, apenas modifica o estado atual dos arquivos no seu diretório de trabalho.

Se você modificou um arquivo, mas ainda não fez `git add`, é possível descartá-lo com:

```
git restore nome_do_arquivo
```

Se você já executou `git add`, mas deseja remover o arquivo da staging area sem apagar as modificações feitas nele, utilize:

```
git restore --staged nome_do_arquivo
```


Nesse caso, o arquivo continua modificado no working directory, mas deixa de estar preparado para commit.

Git commit --amend

Já vimos que varios comandos gits possuem parametros diferentes que alteram as suas funcionalidades. Uma delas é o `--amend` para o git commit

O comando `git commit --amend` serve para alterar o último commit feito. Ele permite:

- Corrigir a mensagem do último commit
- Adicionar arquivos que você esqueceu
- Ajustar pequenas mudanças sem criar um novo commit

CUIDADO: Evite usar `--amend` em commits que já foram enviados para o repositório remoto (`git push`). Isso porque você estará alterando o histórico, e outras pessoas podem ter baseado trabalho naquele commit.

Se já tiver feito push, você precisaria usar:

```
git push --force
```

E isso pode causar problemas em equipe.

Exemplos:

Se esqueceu um arquivo, ou realizou uma pequena correção nele:

```
git add arquivo_esquecido.cpp
git commit --amend
```

Se quiser alterar a mensagem:

Pelo editor de texto:

```
git commit --amend
```

Direto pelo terminal:

```
git commit --amend -m "Nova mensagem corrigida"
```

Reescrevendo o histórico

O comando `git reset` é utilizado para mover o ponteiro da branch atual para outro commit. Dependendo da opção utilizada, ele pode alterar apenas a staging area, o working directory ou até mesmo reescrever o

histórico de commits.

Diferente do `git restore`, que atua apenas sobre arquivos específicos, o `git reset` atua sobre commits e pode modificar o estado da branch como um todo.

Removendo arquivo da staging area

Uma forma comum de utilizar o `git reset` é para remover arquivos da staging area, funcionando de maneira semelhante ao `git restore --staged`.

```
git reset nome_do_arquivo
```

Resetando commits

O `git reset` também pode ser usado para voltar a branch para um commit anterior.

```
git reset --soft <hash-do-commit>
```

O parâmetro `--soft` move o ponteiro da branch para o commit indicado, mas mantém todas as alterações na staging area. É como se o commit nunca tivesse acontecido, mas as modificações continuassem preparadas para um novo commit.

```
git reset --mixed <hash-do-commit>
```

O modo `--mixed` é o padrão. Ele move o ponteiro da branch e remove os arquivos da staging area, mas mantém as alterações no working directory.

```
git reset --hard <hash-do-commit>
```

O modo `--hard` move o ponteiro da branch e descarta completamente todas as alterações posteriores ao commit indicado, tanto da staging area quanto do working directory.

Esse é o modo mais perigoso, pois as alterações são perdidas definitivamente.

ATENÇÃO

O `git reset` reescreve o histórico da branch, alterando o ponteiro para um commit anterior. Por isso, ele não deve ser utilizado em branches que já foram compartilhadas com outras pessoas, pois pode causar conflitos e inconsistências no repositório remoto.

- `git restore` trabalha com arquivos
- `git reset` pode alterar commits
- `git reset --hard` pode apagar alterações

Desfazendo alterações

O comando `git revert` é utilizado para desfazer as alterações introduzidas por um commit específico. Diferente do `git reset`, ele não remove commits do histórico. Em vez disso, ele cria um novo commit que desfaz as modificações feitas anteriormente.

Isso significa que o histórico do projeto é preservado, tornando o `git revert` uma opção segura para branches que já foram compartilhadas com outras pessoas.

Primeiro, identificamos o commit que desejamos desfazer utilizando o `git log`. Em seguida, executamos:

```
git revert <hash-do-commit>
```

Ao executar esse comando, o Git cria automaticamente um novo commit que aplica as alterações inversas do commit selecionado. Caso o commit revertido tenha modificado vários arquivos, todos eles serão revertidos nesse novo commit.